

THY IST Hub Tarife Optimizasyonu — model

Matematiksel Model Dokümanı — Çalışma Sürümü

Bu dosya milestone milestone güncellenir; her milestone DoD'sinin bir parçasıdır. Nihai teslim için PDF'e dönüştürülecek (§5 Teslim Edilecekler, brief). **Kod ile bu dosya arasındaki her tutarsızlık diskalifiye/ağır puan kaybı riskidir** — bir kısıt veya değişken burada değişirse, aynı commit içinde ilgili `src/model/*.py` da güncellenmelidir (ve tersi).

Durum: **M0–M4 tamam** (A–G tam formülasyon, `build_model_m4`, `main.py`'ye bağlı, CLI end-to-end `valid=True`, fixture `objective=668.75`). **M5/M5c/M5d/M5e tamam** — full-data'da doğrulanmış `objective_value` henüz yok; veri v2 entegre (Elapsed-türevli `K_od/R_o`, VARSAYIM-14/15). **M5f kapanışı** (`docs/CLOSING_PLAN.md`, **Kapı-0...6**) — **Kapı-0...5 TAMAM**: KARAR-0 (E1 koşullu aktivasyon, VARSAYIM-16) + KARAR-0b (E2 statik-imkânsız muafiyet, VARSAYIM-17) uygulandı, fixture HER İKİ modda da 668.75 (değişmedi); Kapı-2 full-data'da ölçtü (baseline E1 690→296); Kapı-3 kampanyası (elastik+warm-start → LNS → çoklu-bileşen LNS) TÜKENDİ, **Branch B kesinleşti** (full-data'da doğrulanmış `objective_value` YOK — sekiz bağımsız kanıt yönü, `ASSUMPTIONS.md` VARSAYIM-12 GÜNCELLEME 5); Kapı-4 (yalnız Branch A) uygulanamaz, atlandı; Kapı-5 üretim merdiveni (`main.py --full-data`, ihlalli tarife ASLA yazmama garantisi) GERÇEK full-data'da uçtan uca doğrulandı (`docs/STATUS.md`). Kapı-6 (teslimat) sürüyor — bkz. `docs/report.md`, `docs/traceability.md`.

1 • Kümeler

Sembol	Tanım	Kaynak
S	Dış istasyonlar (outstation) kümesi	O&D tablosu <code>Dep1 / Arr2</code> kolonlarından türetilir
F	TK uçuş numaraları	O&D tablosu <code>F1No1 / F1No2</code>
R	Uçuş örnekleri $r = (f, h)$ — bir uçuş numarasının belirli bir gündeki gerçekleşmesi	O&D tablosu (flno, gün) çiftleri
R^{var}, R^{fix}	Ayarlanabilir / sabit uçuş örnekleri (Standard: $R^{var} = R$, config'ten override edilebilir)	<code>config.adjustable_set</code>
H	Gün indeksi kümesi, $h \in \{1, \dots, 7\}$	O&D tablosu <code>Gün</code> kolonu
Π	Aday bağlantılar $\pi = (r_1, r_2)$: r_1 TK inbound (o→IST), r_2 TK outbound (IST→d), aynı gün h	<code>src/candidates/generate.py</code> — TK inbound×outbound cross-product, $[L, U]$ fizibilite kapısıyla budanmış (Modül-3 kapı 1)
$O-D$	Sıralı (o, d) pazar çiftleri, $o \neq d$	Yolcu Verisi tablosu

2 • Parametreler

Sembol	Tanım	Kaynak / üretim
ρ_{od}	O–D gelir ağırlığı	Yolcu Verisi_masked.xlsx . Not: 12 duplicate (orig,dest) satırı toplanıyor, 3 eksik-dest satırı reddediliyor — bkz. ASSUMPTIONS.md
L, U	Bağlantı boşluğu alt/üst sınır (Standard: 60, 300 dk)	config
τ_o	Minimum yer süresi (Standard: 45 dk)	config
$K_{od} = T_o^{IB} + T_d^{OB}$	Pazar bazlı yolculuk sabiti	src/data/block_times.py::get_journey_constant . v2 (VARSAYIM-14/15, M5e): ElapsedTime1 / ElapsedTime2 kolonları mevcutsa (organizatör 2026-07-09 paketi), K_{od} =market-bazlı medyan(elapsed1+elapsed2), TÜM TK satırları dahil ($[L, U]$ ’dan BAĞIMSIZ — gap artık K_{od} ’a hiç girmiyor). Kolonlar YOKSA (ör. fixture’ın kolonsuz varyantı) mevcut LS-yolu (geçerli-boşluklu medyan) DEĞİŞMEDEN korunur.
$R_o = T_o^{IB} + T_o^{OB}$	İstasyon bazlı rotasyon sabiti	src/data/block_times.py::get_rotation_constant . v2: istasyon-bazlı medyan(Elapsed1)+medyan(Elapsed2) — doğrudan gözlem, LS/ridge YOK. Kolonlar yoksa bipartite least-squares, shift-invariant (bkz. tests/unit/test_block_times.py ve ASSUMPTIONS.md VARSAYIM-15)
$T_{od,h,k}^{comp}, N_{od,h}$	Rakip k ’nin en iyi (min) yolculuk süresi, pazardaki distinct rakip (taşıyıcı) sayısı	src/data/competitors.py::derive_rival_best_times — bir “rakip” TEK BİR TAŞIYICI (Cr1); o taşıyıcının o (o,d,h)’deki TÜM itineraryleri min’e konsolide edilir (ASSUMPTIONS.md VARSAYIM-4)
b_{od}	Taban (baseline) sıralama	src/data/ranking.py::derive_b_od — r’nin KENDİ formülünün baseline (optimizasyon-öncesi) zamana uygulanmış hali: $b_{od} = \max(1, N_{od} - \text{baseline’da yenilen rakip sayısı})$, D’nin $AYNI \leq$ kuralıyla
$W_j^{(c)} = 2^{-(j-1)}$	Azalan getiri slot ağırlığı (formül, veri değil)	Brief §3.1
$W^{(r)}(N, b, r)$	Sıralama ödülü lookup	change_ranking_input.xlsx (negatif değerler içerebilir)
X_{dev}, α, Γ	Gün-içi sapma / yönsel denge / JT farkı eşikleri (Standard: 15, 0.20, 30)	config
bucket boyutu, kapasite (kalkış/varış)	10 dk, 10/15 (Standard)	config

3 · Karar Değişkenleri (M1–M2)

Sembol	Tanım	Domain
t_r^{arr}, t_r^{dep}	Uçuş örneği r ’nin IST varış/kalkış saati, tam sayı dakika (epoch: veri kümesindeki en erken tarihin gece yarısından itibaren)	Integer; $r \in R^{fix}$ için <code>.fix()</code> ile sabitlenir (aynı kod yolu, iki ayrı dal yok)
x_π	Bağlantı π sunuluyor mu	Binary — serbest değil , B ile türetilmiş göstergedir (aşağı bakınız)

Sembol	Tanım	Domain
y_π	B'nin backward-reifikasyonunda hangi taraftan ($< L$ vs $> U$) ihlal edildiğini seçen yardımcı switch	Binary, yalnızca $x_\pi = 0$ iken anlamlı
gap_π	$t_{r_2(\pi)}^{dep} - t_{r_1(\pi)}^{arr}$ (eşitlikle tanımlı)	Integer
$s_{o,d,h,j}$	Modül-5 slot göstergesi, $j = 1, \dots, J_{max}(o, d, h)$	$[0, 1]$ sürekli — $J_{max}(o, d, h)$ = $ \Pi_{o,d,h} $ (veri-türetilmiş, config sabiti değil)
$beat_{\pi,k}$	Bağlantı π , rakip k 'yı yeniyor mu	Binary — forward-only zorlama (aşağı bakınız)
$beaten_{o,d,h,k}$	Rakip k , o (o,d,h) pazarında yenildi mi (OR-aggregation)	Binary
$onehot_{o,d,h,r}$	Güncellenen sıralama $r_{o,d,h}$ 'nin one-hot göstergesi, $r = 1, \dots, N_{o,d,h}$	Binary

Zaman temsili kararı: integer domain (continuous değil). Gerekçe: B'nin backward-reifikasyonu ($L - 1, L$) ve ($U, U + 1$) epsilon-bantları kullanır; continuous zamanla bu bantlar TÜM meşru (kesirli dakikalı) tarifeleri modelin erişemeyeceği bir “yasak bölge” haline getirirdi. Veri zaten dakika hassasiyetinde olduğundan integer domain kayıpsız (bkz. `tests/solve/test_m1_constraints_b.py::test_integer_boundary_*`).

Uçuş-örneği paylaşımı: birden fazla π aynı r 'yi referans edebilir (ör. bir inbound uçuş birden çok outbound ile eşleşebilir) — bu yüzden t_r^{arr}, t_r^{dep} ROL-namespaced $r_{id} = ("IB"|"OB", flno, h)$ ile indekslenir (aynı `flno` 'nun hem inbound hem outbound rolünde görünebildiği gerçek veride 26 örnek doğrulandı — namespace olmadan bu iki farklı zaman değeri aynı Pyomo Var anahtarı altında çakışırdı).

4 • Amaç Fonksiyonu (M1 bağlantı-sayısı + M2 sıralama ödülü)

$$\max \underbrace{\sum_{o \neq d} \rho_{od} \sum_h \sum_j W_j^{(c)} \cdot s_{o,d,h,j}}_{\text{connection_reward (M1)}} + \underbrace{\sum_{o \neq d} \rho_{od} \sum_h \sum_r W^{(r)}(N_{od,h}, b_{od}, r) \cdot onehot_{o,d,h,r}}_{\text{ranking_reward (M2)}}$$

`model.connection_reward` ve `model.ranking_reward` ayrı Pyomo Expression'lar olarak loglanır (amaç bileşen bileşen izlenebilir olsun diye).

5 • Kısıtlar

B — Bağlantı Uygunluğu (M1)

Doğruluk argümanı: $x_\pi = 1$ ancak ve ancak $gap_\pi \in [L, U]$. Tek-yönlü (yalnızca $x = 1 \Rightarrow gap \in [L, U]$) yeterli DEĞİL — E1/E2 (M4) devreye girince solver'a “gerçekten geçerli bir bağlantıyı gizle” motivasyonu doğar (dengesizlik/JT-farkı cezasından kaçınmak için). Backward yön ($gap \in [L, U] \Rightarrow x = 1$) bu açığı kapatır. Standart 4-kısıtlı interval reifikasyonu, yardımcı y_π ile:

$$\begin{aligned}
gap_\pi &\geq L - M_\pi^1(1 - x_\pi) & (\text{forward-lower}) \\
gap_\pi &\leq U + M_\pi^2(1 - x_\pi) & (\text{forward-upper}) \\
gap_\pi &\leq (L - 1) + M_\pi^3(x_\pi + y_\pi) & (\text{backward-below}) \\
gap_\pi &\geq (U + 1) - M_\pi^4(x_\pi + (1 - y_\pi)) & (\text{backward-above})
\end{aligned}$$

Big-M türetimi (`src/model/big_m.py`) — her adayın KENDİ achievable range'inden ($gap_\pi \in [gap_\pi^{lo}, gap_\pi^{hi}]$, bağımsız hareket eden t^{arr}, t^{dep} 'in interval-subtraction'ı), global sabit değil:

$$\begin{aligned}
M_\pi^1 &= \max(0, L - gap_\pi^{lo}) & M_\pi^2 &= \max(0, gap_\pi^{hi} - U) \\
M_\pi^3 &= \max(0, gap_\pi^{hi} - (L - 1)) & M_\pi^4 &= \max(0, (U + 1) - gap_\pi^{lo})
\end{aligned}$$

Model kurulumunda her M otomatik olarak ≤ 1440 **assert edilir** (`MAX_ALLOWED_BIG_M`); aşılsa `ValueError` ile durur (plan'ın kendi Big-M disiplini, market-bazlı sıkılaştırmayı M6'ya ertelemek yerine M1'den itibaren uygulanıyor — daha basit VE daha sıkı).

Adjustable window VARSAYIM (`ASSUMPTIONS.md` VARSAYIM-3): $w = 180$ dk (720 değil) — w büyüdükçe Big-M $O(4w)$ mertebesinde büyüyor; $w = 720$ bazı adaylarda $M > 1440$ 'a çıkarıyordu.

C — Bağlantı Seçimi ve Azalan Getiri (M1)

Doğruluk argümanı: $s \in [0, 1]$ sürekli, $\sum_j s_j = \sum_\pi x_\pi$, $s_{j+1} \leq s_j$ (monoton), ve $W_j^{(c)}$ kesin azalan olduğundan, optimal her zaman düşük- j (yüksek ağırlıklı) slotları önce doldurur — sabit toplamı korurken ağırlığı düşük- j 'den yüksek- j 'ye kaydırmak ödülü asla artıramaz. Bu, LP optimumunu binary deklare etmeden integer köşeye oturtur (daha az integer değişken $\rightarrow$ Performans, doğrulandı: `test_slot_values_settle_at_integer_vertices_not_fractional`).

$$\sum_j s_{o,d,h,j} = \sum_{\pi \in \Pi_{o,d,h}} x_\pi \quad s_{o,d,h,j+1} \leq s_{o,d,h,j}$$

D — Rakip Yenme ve Sıralama (M2)

beat reifikasyonu — doğruluk argümanı: $beat_{\pi,k} = 1 \iff J_\pi \leq T_k^{comp}$ ($J_\pi = K_{od} + gap_\pi$). Gerçek `change_ranking_input.xlsx` tablosu **monotonik** (sabit (N, b) için r arttıkça W hiç artmıyor — 820 grupta 0 ihlal, `tests/unit/test_ranking.py`). Bu doğruyken **tek-yönlü (forward)** zorlama yeterli:

$$J_\pi \leq T_k^{comp} + M_{\pi,k}^{fwd}(1 - beat_{\pi,k}) \quad beat_{\pi,k} \leq x_\pi$$

Backward yön ($J_\pi \leq T_k^{comp} \Rightarrow beat_{\pi,k} = 1$) gerekmiyor çünkü W monoton azalan olduğundan under-claim (yenilen bir rakibi yenilmemiş göstermek) objektifi ASLA artıramaz — solver'ın bunu yapma motivasyonu yok. **Monotonluk bozulursa** (`is_ranking_monotonic` `False` dönerse) sistem otomatik `monotonic=False` moda geçer, tam bidirectional forcing eklenir (ikisi de kodda hazır, `src/model/constraints_competition.py::add_d_constraints`).

$$M_{\pi,k}^{fwd} = \max(0, J_\pi^{hi} - T_k^{comp}), \quad J_\pi^{hi} = K_{od} + gap_\pi^{hi}$$

M5c D-folding (docs/lp_anatomy.md): $beat_{\pi,k}$ 'ye HER ZAMAN bir Binary değişken açılmaz — $J_{\pi}^{hi} \leq T_k^{comp}$ ise π pencerenin TAMAMINDA rakip k 'yı yeniyor (veri-gerçeği, $beat \equiv x_{\pi}$ 'ye katlanır, değişken üretilmez); $J_{\pi}^{lo} > T_k^{comp}$ ise HİÇBİR zaman yenmiyor ($beat \equiv 0$ 'a katlanır). Bu, monoton VE bidirectional-fallback modlarının İKİSİNDE de geçerli (adayın penceresine dair bir veri-gerçeği, zorlama yönünden bağımsız) — değişken/kısıt eklemek yerine ELEME, LP gevşemesini de sıkılaştırır (fractionality full-data'da $beat$ için %14→katlanan çiftler için 0).

OR-aggregation ($beaten_{o,d,h,k}$) HER ZAMAN iki yönlü — monotonluktan bağımsız yapısal bir gereklilik (iç tutarlılık):

$$beaten_{o,d,h,k} \geq beat_{\pi,k} \quad \forall \pi \quad beaten_{o,d,h,k} \leq \sum_{\pi} beat_{\pi,k}$$

$$r_{o,d,h} = N_{o,d,h} - \sum_k beaten_{o,d,h,k}$$

Rank one-hot — kritik düzeltme (ultrathink + CLI end-to-end testyle yakalandı): $r = N - beaten$ formülü $[0, N]$ üretebilir ama gerçek tabloda r asla 0 değil (min gözlenen $r = 1$, tüm N için doğrulandı). İlk tasarımda linking EŞİTLİK'ti ($\sum(r \cdot onehot_r) == r_{\{o,d,h\}}$) — bu, solver $beaten=N$ 'e ulaştığında $r = 0$ gerektiriyordu ama $onehot$ 'un $r = 0$ karşılığı YOKTU, yani solver YAPISAL OLARAK en az bir rakibi bedava olsa bile kasıtlı yenilmemiş bırakmak ZORUNDA kalıyordu (infeasibility tuzağı). **Düzeltilme**: linking EŞİTSİZLİK yapıldı:

$$\sum_j j \cdot onehot_{o,d,h,j} \geq r_{o,d,h} \quad \sum_j onehot_{o,d,h,j} = 1$$

$onehot$ 'un kendi domain'i $[1, N] + W$ 'nin monotonluğu, optimizier'ı otomatik olarak $r = \max(1, N - beaten)$ 'e oturtuyor (C'nin slot argümanı ile AYNI mantık — $\max()/\min()$ lineerleştirilmesi gerekmedi).

Under-claim toleransı: forward-only forcing claimed_beaten'i HER ZAMAN actual_beaten'in alt kümesi yapar (over-claim yapısal olarak imkansız) — bu yüzden src/validate/independent_validator.py under-claim'i violation olarak İŞARETLEMEZ (ödül asla şişirilmiyor, sadece eksik bilgi). Ayrıntı: tests/fixtures/README.md “M2 eki”, docs/decisions.md 2026-07-09.

A — Zaman Sınırları ve Uçak Rotasyonu (M3)

Doğruluk argümanı: “Aynı uçak IST→o→IST iki bacaklı bir görevi gerçekleştirir” — KOŞULSUZ bir operasyonel kural (x_{π} 'den bağımsız, hangi bağlantıların sunulduğundan etkilenmez, sadece fiziksel uçak kısıtı). $R_o = T_o^{IB} + T_o^{OB}$ zaten TEK sağlayıcı çağrısıyla geliyor (block_times.get_rotation_constant):

$$t_{(IB, flno_{dönüş}, h)}^{arr} \geq t_{(OB, flno_{gidiş}, h)}^{dep} + R_o + \tau_o$$

Big-M **gerekmiyor** (koşulsuz eşitsizlik, reifikasyon değil).

Kapsam sınırlaması (ASSUMPTIONS.md VARSAYIM-5): yalnızca Pair grubu içindeki ARDIŞIK (Orig==IST → sonra Dest==IST) bacak çiftlerine uygulanır. Gerçek veride 707 Pair grubundan 657'si tam 2-üyelidir (doğrudan kapsanıyor); 50'si 3+ üyelidir, bazıları IST'e değmeyen ara bacaklar içeriyor (ör. IST→MEX→CUN→IST) — bu ara bacaklar modelin t^{arr}/t^{dep} değişken kapsamı DIŞINDA (yalnızca IST-tarafı zamanlar modelleniyor), o gruplar için kısıt EKSİK kalıyor (bilinçli, dokümanla edilmiş sınırlama).

Eşleştirme kuralı — BASELINE KRONOLOJİSİ (M5 VARSAYIM-10): yukarıdaki formülün $f_{lno_{gidiş}}/f_{lno_{dönüş}}$ eşleştirmesi (hangi OB kalkışının hangi IB varışıyla eşleştiği) M3'te "AYNI GÜN" varsayımıyla yapılıyordu. Gerçek veride bu YANLIŞ: R_o genellikle SAATLER mertebesinde (uzun menzilde 6-21 saat) olduğundan, aynı gün'ün IB'si çoğu zaman GERÇEK dönüş bacağı değil, TAMAMEN ALAKASIZ bir rotasyon. Full data'da 1496 rotasyon-çift örneğinin 818'i (%54.7) baseline'da uzlaştırılmaz, %45.3'ü KRONOLOJİK OLARAK TERS (IB varışı OB kalkışından ÖNCE) çıktı.

Düzeltilme (`src/model/rotation_matching.py::match_rotation_legs`): her OB kalkışının partneri, BASELINE saat-of-day'e göre KENDİSİNDEN SONRAKİ EN YAKIN IB varışı — dairesel (Gün haftalık tekrarlanan desen, Gün=7'den Gün=1'e sarar), açgözlü ve BİREBİR. Kısa menzilde (aynı gün içinde döner) bu kural zaten "aynı gün" ile AYNI sonucu verir — M3 davranışı korunur. Sarma (Gün=7→Gün=1) durumunda kısıtın ham epoch kıyası bir HAFTA (`WEEK_PERIOD_MIN=10080`) ileri kaydırılır, aksi halde önceki haftanın (yanlış) değeriyle kıyaslanır:

$$t_{ib_{gun}}^{arr} + \text{week_offset} \geq t_{ob_{gun}}^{dep} + R_o + \tau_o$$

Kalıcı istisna — VARSAYIM-11: doğru eşleştirmeye bile full data'nın %24.3'ü (382/1571) KENDİ en-iyi-durum ayarlamasında bile uzlaştırılmaz (G'nin TK2841 durumuyla AYNI yapısal senaryo). Bu çiftler, her bacağın KENDİ $[t_{lo}, t_{hi}]$ penceresi kullanılarak ($t_{hi}^{arr} + \text{week_offset} \geq t_{lo}^{dep} + R_o + \tau_o$ testile) tespit edilip A kısıtından MUAF tutulur (loglanır, sessizce atlanmaz).

Bağımsız doğrulama: `independent_validator.py::_match_rotation_legs_independent` AYNI algoritmayı bağımsız (import etmeden) yeniden uygular, baseline kronolojisini ham TK tablosundan (raporlanan zamanlardan DEĞİL) türetir.

G — Tarife Düzenliliği (M3)

Doğruluk argümanı — referans-zaman formülasyonu: brief " $\max(t_h) - \min(t_h) \leq X_{dev}$ " istiyor. Naif formülasyon HER GÜN ÇİFTİ için $|t_{h_1} - t_{h_2}| \leq X_{dev}$ kurar ($O(H^2)$ kısıt). Bunun yerine serbest bir referans değişkeni $T_{role, flno}$ tanımlanır, her gün için:

$$T_{role, flno} \leq t_{role, flno, h} \leq T_{role, flno} + X_{dev} \quad \forall h$$

Eşdeğerlik kanıtı: ($\Leftarrow$) Tüm t_h aynı X_{dev} -genişlikli pencerede ise herhangi iki günün farkı en fazla X_{dev} . ($\Rightarrow$) $\max(t_h) - \min(t_h) \leq X_{dev}$ ise $T = \min(t_h)$ seçilir; o zaman tüm $t_h \leq \max(t_h) \leq \min(t_h) + X_{dev} = T + X_{dev}$ otomatik sağlanır, ve $t_h \geq \min(t_h) = T$ zaten tanım gereği. Formülasyon TAM eşdeğer (gevşek değil), $O(H)$ kısıtla ifade ediyor (H=gün sayısı) — daha sıkı LP gevşetmesi, daha küçük branch-and-bound ağacı.

Kritik düzeltme — gün-içi normalizasyon (ilk solve denemesinde infeasibility olarak yakalandı):

$t_{role, flno, h}$ 'nin epoch değeri TEK bir GLOBAL çapaya göre (`compute_epoch_anchor` , tüm veri kümesinin en erken tarihinin gece yarısı) — farklı h (gün) değerleri farklı TAKVİM günlerine denk geldiğinden, epoch değerleri arasında ~1440dk'lık (bir gün) fark vardır, saat-of-day AYNI olsa bile. Formülü ham epoch değerlerine doğrudan uygulamak, saat-of-day'i tamamen uyumlu bir tarifeyi bile ~1440dk'lık SAHTE bir ihlal olarak görür — X_{dev} ile ASLA uzlaştırılmaz, model infeasible olur. **Çözüm:** her $(role, flno, h)$ kendi takvim gününün gece yarısına göre normalize edilir ($t - \text{day_offset}(role, flno, h)$) önce kısıta girer — $T_{role, flno}$ artık "gün-içi referans dakika" temsil eder. `src/model/constraints_operations.py::_day_offsets` , aynı düzeltme `independent_validator.py` 'nin x_{dev} kontrolünde de tekrarlanır (aksi halde GEÇERLİ bir çözüm bile validator tarafından yanlışlıkla reddedildi).

İkinci kritik düzeltme — gece yarısı SARMASI (M4 “G check”): yukarıdaki düzeltme referans noktası olarak GERÇEK gece yarısını (00:00) kullanıyordu. Bu, KENDİ saati gece yarısına YAKIN olan bir uçuş için hâlâ yanlış: 23:55 (gün-ici=1435) ile 00:05 (gün-ici=5) arasındaki GERÇEK fark 10 dakikadır, ama gece-yarısı-çapalı temsilde $|1435 - 5| = 1430$, X_{dev} ile ASLA uzlaştıramayan SAHTE bir ihlal. Çözüm: referans noktası her (role,fno) çifti için, o uçuşun KENDİ baseline saatinin TAM 12 SAAT KARŞISINA kaydırılıyor (`_flight_cut_points`, `saat+720 mod 1440`) — uçuşun ayarlanabilir aralığı (Big-M disiplini gereği hiçbir zaman $\pm 720dk$ 'yı aşamaz) bu yeni sınırdan ASLA taşamaz, sarma sorunu o uçuş için yapısal olarak imkansız hale gelir. Elle doğrulandı ve `test_g_no_false_violation_at_midnight_wraparound` (RED→GREEN) ile kanıtlandı. Validator'ın x_{dev} kontrolü BİREBİR aynı mantığı tekrarlar.

Üçüncü kritik düzeltme — KÜME-BAZLI G (M5 VARSAYIM-9): full-data solve merdiveni tüm adımlarda HIZLI infeasible verdi (bkz. ASSUMPTIONS.md VARSAYIM-9) — kök neden, gerçek TK2841 (TZX→IST) uçuşunun KENDİ baseline tarifesinin bile G'nin koşulsuz (“tüm günler tek grup”) okumasını tatmin edemeyecek kadar günden güne değişmesi (4 günde 03:25, 1 günde 14:10, 645dk fark — $\pm 180dk$ pencere ile en fazla $2 \times 180 + 15 = 375dk$ uzlaştırılabilir, $645 > 375$). Katı okumada uzlaştırılabilir küme kümesi BOŞ — tüm problem infeasible olurdu.

Formel kanıt (1D Helly özelliği, aralıklar için): bir grubun (aynı role,fno'nun gün-örnekleri) tek bir ortak T_{ref} ile uzlaştırılabilmesi

$$\Leftrightarrow \forall h : [baseline_h - w_h, baseline_h + w_h] \cap [T_{ref}, T_{ref} + X_{dev}] \neq \emptyset$$

$$\Leftrightarrow T_{ref} \in \bigcap_h [baseline_h - w_h - X_{dev}, baseline_h + w_h]$$

$$\Leftrightarrow \max_h (baseline_h - w_h) - \min_h (baseline_h + w_h) \leq X_{dev}$$

$$\Leftrightarrow \max(baseline) - \min(baseline) \leq w_{\min-\text{örnek}} + w_{\max-\text{örnek}} + X_{dev} \quad (\text{ÇAP koşulu})$$

Bu bir **ÇAP** (diameter) koşuludur, ARDIŞIK-boşluk (single-linkage) koşulu DEĞİL: 0dk/300dk/600dk'lık üç nokta ($w = 0$, $X_{dev} = 310$) ardışık bakışta ($300 \leq 310$ her komşu çift için) tek kümeye yanlışlıkla birleşirdi, ama 0 ile 600 arasındaki GERÇEK 600dk'lık fark 310'u aşıyor — ortak bir 310dk'lık pencereye asla sığmazlar.

Karar: G artık FLIGHT bazında değil KÜME bazında uygulanıyor

(`src/model/day_clustering.py::cluster_flight_days`). Algoritma: (1) dairesel eksende (mod 1440) EN BÜYÜK boşluktan kes (rastgele bir kesim noktasının gerçek bir kümeyi bölmesini önler — gece yarısı sarmasıyla AYNI motivasyon, `_flight_cut_points` 'le tutarlı), (2) doğrusallaştırılmış diziyi soldan sağa AÇGÖZLÜ ÇAP taraması: her nokta KÜME BAŞLANGICINA (sıralı taramada her zaman en küçük tod'lu, ilk eklenen üye) göre kontrol edilir, ARDIŞIK ÖĞEYE göre DEĞİL. Her döndürülen kümenin çap koşulunu sağladığı invariant olarak assert edilir. Veri-türetilmiş ve GENEL (belirli bir uçuş numarasına özel hiçbir şey hardcoded edilmiyor — gizli test seti güvenliği).

$$t_{f,h} \in [T_{ref}^{(f,c)}, T_{ref}^{(f,c)} + X_{dev}] \quad \forall h \in c$$

Tüm günler zaten uzlaştırılabilirse (yaygın durum — gerçek veride 460/461 çok-günlü IB uçuşu, 476/476 OB uçuşu) TEK küme oluşur = M3 davranışı DEĞİŞMEDEN korunur; yalnızca TK2841 gibi yapısal aykırı değerler kendi tekil kümelerine ayrılır. `independent_validator.py::_cluster_flight_days_independent`

AYNI algoritmayı bağımsız (import etmeden) yeniden uygular — yalnızca AYNİ kümenin İÇİNDEKİ raporlanan zamanlar X_{dev} 'e tabi, kümeler arası karşılaştırma yapılmaz.

Rol-ayırımı: aynı f_{lno} hem IB hem OB rolünde görünebilir (M1'de 26 gerçek örnek doğrulandı) — her rol KENDİ ayrı $T_{role, f_{lno}}$ 'suna sahip (brief “kalkış VE varış saatleri” diyerek zaten ayrı ele alıyor). Yalnızca 2+ farklı günde modelde bulunan (role, f_{lno}) çiftleri için kısıt kurulur.

E1 — Yönsel Sayı Dengesi (M4, KARAR-0 koşullu aktivasyon M5f)

Doğruluk argümanı: $n_{fwd} = \sum_{\pi} x_{\pi}$ (o,d,h pazarı), $n_{bwd} = \sum_{\pi} x_{\pi}$ (d,o,h pazarı) zaten LINEER ifadeler (C'nin market-grouping'inden) — reifikasyon/Big-M GEREKMİYOR ana toplamalar için. Yalnızca HER İKİ yönde de candidate'ı olan pazar çiftlerine uygulanır (tek-yönlü pazarları zorlamak modelin KAPSAM sınırlamasından kaynaklanan yapay bir kısıtlama olurdu).

KARAR-0 (ASSUMPTIONS.md VARSAYIM-16, docs/CLOSING_PLAN.md): varsayılan

`activation="conditional"` modda kısıt yalnızca HER İKİ yön de AKTİFKEN (≥ 1 sunulan bağlantı)

bağlayıcı — E2'nin zaten var olan aktivasyon göstergesi a_{fwd}/a_{bwd} (`model.a_dir`, “en az bir aday sunuldu mu”) yeniden kullanılır (satır/binary ekonomisi, E2 bu fonksiyondan ÖNCE çalışmış olmalı):

$$n_{fwd} - n_{bwd} \leq \alpha(n_{fwd} + n_{bwd}) + M_{pair}(2 - a_{fwd} - a_{bwd})$$

$$n_{bwd} - n_{fwd} \leq \alpha(n_{fwd} + n_{bwd}) + M_{pair}(2 - a_{fwd} - a_{bwd})$$

$M_{pair} = (1 - \alpha) \cdot \max(|\Pi_{fwd}|, |\Pi_{bwd}|)$ (`src/model/big_m.py::derive_e1_pair_big_m`, pazar-çifti bazlı, adayların KENDİ sayısından türetilir — global sabit YOK, doğası gereği küçük). $a_{fwd} = a_{bwd} = 1$ olduğunda (M terimi 0) formül **literal/unconditional** okumayla BİREBİR aynıdır — `activation="unconditional"` bu eski okumayı duyarlılık analizi için ayrı bir bayrakla korur (`src/config/standard.yaml::e1_activation`). İki yön de pasif $\rightarrow 0 \leq 0$ her ikisinde, otomatik sağlanır (her iki modda).

Kritik davranışsal etki (literal/unconditional modda): B'nin “ $gap \in [L, U] \Rightarrow x = 1$ ZORUNLU” kuralı, E1'i sağlamanın YEGANE yolunun bir bağlantıyı KISMEN gizlemek değil (B tarafından yapısal olarak engelleniyor), zamanı kaydırıp gap 'i $[L, U]$ dışına iterek bağlantıyı TAMAMEN ÖLDÜRMEK olduğu anlamına gelir. Küçük/asimetrik pazarlarda bu, literal E1'i bir **amaç bastırıcı** yapar: solver tek bir fazla bağlantıyı dengelemek yerine TÜM pazarı sıfırlamayı tercih edebilir. Ampirik kanıt (ASSUMPTIONS.md VARSAYIM-16): full-data'da ulaşılan HER noktada E1 fazlalık oranı sabit $0.800 = 1 - \alpha$ — ihlallerin ~tamamı tek-yön-sıfır vakası, koşullu aktivasyon bunu kaynağında ortadan kaldırıyor.

`src/model/constraints_balance.py::e1_diagnostics` bu davranışı post-solve izler (her pazar çifti için $n_{fwd}/n_{bwd} +$ sıfıra-inme bayrağı, rapora girecek metrik).

E2 — Yön-Arası Seyahat Süresi Farkı (M4)

Brief: her iki yönde de en az bir bağlantı SUNULUYORSA, iki yönün EN İYİ (minimum) seyahat sürelerinin farkı Γ dakikayı aşmamalı. Bu, D'nin OR-aggregation'ından (herhangi biri mi) yapısal olarak FARKLI bir talep: pazarın “en iyi” J değeri SEÇİLEBİLİR bir alt kümenin minimumu — MIP'in doğal bir $\min(\cdot)$ operatörü yok, bu yüzden bir **argmin sandviç** gerekiyor.

Değişkenler (her (o, d, h) pazarı için): - $a_{dir} \in \{0, 1\}$: pazar aktif mi (en az bir π sunuluyor mu) — D'nin `beaten_k` deseniindeki OR-aggregation ile BİREBİR aynı yapı:

$$a_{dir} \geq x_\pi \quad \forall \pi \quad a_{dir} \leq \sum_{\pi} x_\pi$$

- $w_\pi \in \{0, 1\}$ (her candidate için): π pazarın argmin'i olarak SEÇİLDİ mi:

$$w_\pi \leq x_\pi \quad \forall \pi \quad \sum_{\pi} w_\pi = a_{dir}$$

($a_{dir} = 1$ ise tam bir π seçilir; $a_{dir} = 0$ ise hiçbirisi.) - $J_{best} \in [JD_{lo}, JD_{hi}]$: pazarın iddia edilen minimum seyahat süresi (JD_{lo}/JD_{hi} = pazardaki TÜM candidate'ların $J_\pi = K_{od} + gap_\pi$ achievable aralığının min/max'ı — `derive_e2_candidate_big_ms`). **Sürekli (Reals), Integers DEĞİL** — M5d fix (VARSAYIM-13, ASSUMPTIONS.md): argmin sandviçi bir aday seçildiğinde J_{best} 'i TAM olarak o adayın J_π 'sine eşitler (Big-M terimleri sıfırlanır); K_{od} tahmini (LS-estimate, VARSAYIM-8) neredeyse HER ZAMAN kesirli (%99.3, full-data'da 803 pazardan 797'si) — J_{best} Integers olsaydı bu pazarların HERHANGİ biri argmin olarak seçildiğinde KOŞULSUZ infeasible olurdu. J kavramsal olarak zaten sürekli bir büyüklük, Integers M4'ten kalma yanlış bir varsayımı.

Sandviç kısıtları (her candidate π için):

$$J_{best} \leq J_\pi + M_\pi^{up}(1 - x_\pi) \quad J_{best} \geq J_\pi - M_\pi^{down}(1 - w_\pi)$$

Doğruluk argümanı (adversarial: solver J_{best} 'i sahte düşük gösteremez): “ $\leq$ ” tek başına HER sunulan candidate için ayrı bir üst sınır koyar ($x_\pi = 1$ iken $M_\pi^{up} = 0$ 'a kadar sıkışabilir, $J_{best} \leq J_\pi$ tam) — bu J_{best} 'i gerçek minimumun ÜSTÜNE hiç çıkaramaz, ama TEK BAŞINA altına indirmeyi de engellemez (pazardaki DİĞER, sunulmayan candidate'ların geniş achievable aralığı J_{best} 'in kendi $[JD_{lo}, JD_{hi}]$ sınırından ASLA taşamayacağı için, w_π olmadan J_{best} pazarın en düşük olası J 'sine — sunulmuş olsun olmasın — serbestçe kayabilirdi). $w_\pi \leq x_\pi + \sum w_\pi = a_{dir}$ solver'ı SADECE fiilen sunulan bir candidate'a $w = 1$ vermeye zorlar; O candidate için “ $\geq$ ” J_{best} 'i O'nun KENDİ J_π 'sinin altına inmekten yapısal olarak alıkoyar. İki yönün KESİŞİMİ (aynı anda hem üstten hem alttan sıkışan J_{best}) yalnızca gerçek $\min(J_\pi : x_\pi = 1)$ noktasında FEASIBLE'dır — sahte-düşük bir J_{best} iddiası, seçilen w_π 'nin kendi J_π değeriyle ÇELİŞEN bir “ $\geq$ ” kısıtına çarpar (bkz.

`tests/solve/test_m4_constraints_e2.py::test_e2_sandwich_cannot_fabricate_jbest_below_true_min`, pazarın diğer bir candidate'ının çok düşük achievable aralığı bilerek M_π^{down} 'ı ayartmaya çalışıyor ve başarısız oluyor).

M5c w/a_{dir} fold (docs/lp_anatomy.md öncelik #2, ultrathink): bir pazar-yönünün TEK candidate'ı ($|\pi \in \text{group}| = 1$) varsa, $a_{dir} \geq x_\pi$ ile $a_{dir} \leq \sum x_\pi = x_\pi$ birlikte $a_{dir} = x_\pi$ 'yi cebirsel olarak ZORUNLU kılar; $\sum w_\pi = a_{dir}$ tek terimli olduğundan $w_\pi = a_{dir} = x_\pi$ de ZORUNLU. Ayrı bir binary hiçbir yeni bilgi taşımıyor — D-folding'deki AYNI desen (gerçek bir seçim özgürlüğü yoksa değişken ELE). Full-data'da singleton pazar-yönleri grupların %51.4'ü (3935/7656), adayların %21.7'si (3935/18118) — gerçek bir yapısal fold (K-subset'in aksine, bkz. docs/decisions.md 2026-07-10). `model.a_dir / model.w` artık `pyo.Expression` (Var DEĞİL): çoklu-adaylı yönler için gerçek bir arka-plan binary'e (`_a_dir_var / w_var`) sarılır, singleton yönler için doğrudan x_π 'ye çözülür — dışarıya (testler, `e1_diagnostics`) HER ZAMAN aynı `pyo.value(model.a_dir[key])` arayüzünü sunar, katlanmış olsun ya da olmasın.

E2'nin kendisi (fwd/bwd pazar çifti, aynı gün):

$$J_{best}^{fwd} - J_{best}^{bwd} \leq \Gamma + M_{pair}(2 - a_{fwd} - a_{bwd})$$

$$J_{best}^{bwd} - J_{best}^{fwd} \leq \Gamma + M_{pair}(2 - a_{fwd} - a_{bwd})$$

$M_{pair} = \max(0, JD_{hi}^{side} - JD_{lo}^{other} - \Gamma)$ (`derive_e2_pair_big_m`) tam olarak “iki yönün KENDİ ilan edilmiş J_{best} değişken sınırlarının izin verdiği en kötü fark $-\Gamma$ ” — faktör 1 (tek yön aktif) veya 2 (hiçbiri aktif değil) olduğu an kısıt J_{best} ’in kendi bounds’u tarafından zaten sağlanan bir eşitsizliğe gevşer (kanıt: $J_{best}^{side} \leq JD_{hi}^{side}$ ve $J_{best}^{other} \geq JD_{lo}^{other}$ HER ZAMAN, dolayısıyla fark hiçbir zaman $JD_{hi}^{side} - JD_{lo}^{other} = \Gamma + M_{pair}$ ’i aşamaz — bu Big-M’nin M’sinin “tam gevşetme” tanımının doğrudan sonucu). Yalnızca $a_{fwd} = a_{bwd} = 1$ (4 satırlık tablonun son satırı) iken kısıt gerçekten BAĞLAYICI hale gelir.

4 satırlık aktivasyon tablosu — her satır ayrı test (`tests/solve/test_m4_constraints_e2.py`):

a_{fwd}	a_{bwd}	Beklenen davranış
0	0	E2 tamamen gevşek — J_{best} ’ler kendi $[JD_{lo}, JD_{hi}]$ aralığında serbest
1	0	E2 gevşek (karşılaştırılacak bir “diğer yön” yok), ama fwd’in J_{best} ’i yine de DOĞRU (gerçek min) — pasif tarafın varlığı sandviçi bozmuyor
0	1	Simetrik (yukarisının aynası)
1	1	E2 BAĞLAYICI: $ J_{best}^{fwd} - J_{best}^{bwd} \leq \Gamma$ zorunlu

Bağımsız doğrulama: `independent_validator.py`’nin `gamma` kontrolü model kodundan hiç import almadan, SEÇİLMİŞ (offered) bağlantıların raporlanan `gap`’lerinden Python `min()` ile $J_{best}^{fwd} / J_{best}^{bwd}$ ’i yeniden hesaplar (argmin sandviç MIP mekanizmasına ihtiyaç yok — zaten seçilmiş adaylar arasından minimum almak yeterli) ve Γ eşitsizliğini bağımsız olarak assert eder.

KARAR-0b — statik-imkânsız çift muafiyeti (ASSUMPTIONS.md VARSAYIM-17, M5f): $a_{fwd} = a_{bwd} = 1$ olsa BİLE, iki yönün candidate’larının KENDİ $[gap_{lo}, gap_{hi}]$ aralıklarından (seçimden bağımsız, $[L, U]$ ’ya kırılmış) türetilen BEST-CASE J aralıkları Γ ’dan daha fazla ayrıkça, HiÇBİR seçim E2’yi sağlayamaz — `src/model/lns.py::compute_gamma_infeasible_pairs` bu çiftleri schedule-independent olarak tespit eder, `add_e2_constraints` (ve elastik/ folded eşdeğerleri) bu çiftleri E2_PAIRS’ten MUAF tutar (satır hiç kurulmaz) + loglar. Validator aynı testi bağımsız yeniden uygular (`_is_gamma_statically_infeasible` , `src.model` import ETMEZ).

F — Hub Kova/Kapasite Bağlama (M4)

Değişkenler: her ayarlanabilir uçuş örneği (`rol`, `flno`, `gün`) için, PENCERE-ULAŞILABİLİR kovalar üzerinde $z_{r,b}^{dep} / z_{r,b}^{arr} \in \{0, 1\}$ (`derive_window_reachable_buckets` — $[k\Delta, (k+1)\Delta]$ ile $[t_{lo}, t_{hi}]$ ’nin kesiştiği TÜM k ’lar, GÜNÜN TÜM kovaları (144, $\Delta = 10$) DEĞİL — M5 ölçek performansı için kritik bir budama).

Doğruluk argümanı: her örnek $x_{\cdot}$ ’den (bağlantı sunuluyor mu) bağımsız olarak HER ZAMAN hub’da fiziksel bir kovaya denk gelir — bu yüzden $\sum_b z_{r,b} = 1$ KOŞULSUZ (D/E1/E2 gibi $x_{\cdot}$ ’ye bağlı değil, F ARR/DEP_INSTANCES üzerinde çalışır, CANDIDATES üzerinde değil). Kova-zaman bağlama (M5c row-explosion fix, `docs/lp_anatomy.md` öncelik #1): kovalar t ekseninde ARDIŞIK/AYRIK bir bölme olduğundan, t_r Big-M’SİZ, tek bir eşitlikle bijective olarak çözülür:

$$t_r = \sum_b b \Delta z_{r,b} + o_r, \quad o_r \in [0, \Delta - 1] \text{ (yeni tamsayı değişken, offset)}$$

$\sum_b z_{r,b} = 1$ sayesinde toplamda yalnızca SEÇİLEN b 'nin terimi hayatta kalır; t_r 'nin kendi $[t_{lo}, t_{hi}]$ Var bound'u zaten ayrıca uygulandığından, bu eşitlik t_r 'yi seçilen kovanın $[t_{lo}, t_{hi}]$ ile kesişen kısmına pinler — eski per-bucket Big-M ÇİFTİNİN (kova başına lower+upper, örnek başına $\sim 2 \times 37 \approx 74$ satır) ürettiğiyle BİREBİR aynı fizibilite kümesi, ama Big-M GEREKİYOR ve satır sayısı örnek başına 1'e düşüyor. Full-data ölçümü (docs/lp_anatomy.md): F satırları 405,982 $\rightarrow$ $\sim 12,900$ (-%96.8), toplam model satırı 722,947 $\rightarrow$ 329,842 (-%54.4), LP çözüm süresi 194.9s $\rightarrow$ 115.1s, LP objective/LP-tavan oranı DEĞİŞMEDİ (%65.01, eşdeğerliği doğruluyor).

Kapasite (ayrı departure/arrival aileleri, ayrı taban kapasiteler 10/15):

$$\sum_{r: b \text{ ulaşılabilir}} z_{r,b}^{dep} \leq \text{residual}_b^{dep} \quad \sum_{r: b \text{ ulaşılabilir}} z_{r,b}^{arr} \leq \text{residual}_b^{arr}$$

Rezidüel kapasite (VARSAYIM-7, ASSUMPTIONS.md): modelin hiç değişkeni olmayan (kapsam-dışı) TK bacakları kendi HAM baseline zamanında SABİT işgal ettiği kabul edilir ve o kovanın kapasitesinden düşülür (compute_out_of_scope_baselines — tam-tarama precompute, model kurulmadan ÖNCE bir kez; compute_residual_capacity — kapasiteyi düşer, 0'ın altına inmez). A'nın rotasyon kısıtı da AYNI kapsam-dışı-baseline kaynağını paylaşır (edge case: bir Flight Pair alt-çiftinin bacaklarından biri kapsam dışıysa, in-scope bacağa yine de kapsam-dışı ortağın SABİT baseline'ına karşı kısıt kurulur — build_rotation_pairs'in partial_pairs çıktısı).

Bağımsız doğrulama: independent_validator.py'nin bucket_size_min kontrolü, MIP z-binary mekanizmasına hiç ihtiyaç duymadan, reported_times üzerinden doğrudan t//bucket_size_min sayarak kova-doluluğu yeniden hesaplar; kapsam-dışı işgal AYNI mantıkla (raw TK tablosundan) bağımsız olarak türetilir.

Test kapsamı (tests/unit/test_capacity.py , tests/solve/test_m4_constraints_f.py): pencere-ulaşılabilir kova sayısının günün 144 kovasından ÇOK küçük kaldığı doğrulanıyor, kapasite gevşekken bağlayıcı olmadığı, sıkı kapasitede iki adayın FARKLI kovalara zorlandığı (capacity_departure=1 fixture'ı), kapsam-dışı bir uçuşun rezidüel kapasiteyi gerçekten düşürdüğü, ve departure/arrival ailelerinin birbirinden BAĞIMSIZ olduğu (aynı sayısal kova indeksini paylaşırsalar bile rekabet etmiyorlar).

Salt-Fizibilite Modeli (M5c §3, build_feasibility_model)

Amaç: build_model_m4'ün REWARD hesaplama makinesini (C + D) tamamen çıkaran, yalnızca A/B/E1/E2/F/G'yi kuran küçültülmüş bir model — full-data MIP'in kök-düğümde takılı kalması karşısında bir "Plan B" (docs/decisions.md 2026-07-10): HiGHS'in kök-düğüm cut üretimini yavaşlatan hacmin büyük kısmı zaten F'den geliyordu (satır-patlama fix'inden SONRA bile), ama reward'ın KENDİSİ (C'nin slot değişkenleri + D'nin beat/beaten/rank_onehot Big-M zinciri) ayrı bir yük katıyor — bu modelde HİÇBİRİ yok.

Ultrathink (tek paragraf doğruluk argümanı): C (monoton slot, model.s) ve D (rakip yenme + sıralama, model.beat / beaten / rank_onehot) SADECE ödül fonksiyonunun sıralama terimini hesaplamak için var — hiçbir $t_{arr}/t_{dep}/x_{\pi}/gap_{\pi}$ üzerinde bir kısıt KURMUYOR (C'nin s değişkenleri ve D'nin Big-M reifikasyonları yalnızca BİRBİRLERİNE ve x_{π} 'ye bakan, x_{π} 'yi GERİYE doğru kısıtlamayan yardımcı makine — x_{π} sabitken her ikisi de HER ZAMAN tutarlı bir atama kabul eder: s'in monoton/toplam kısıtı $\sum x_{\pi}$ 'ye göre her zaman çözülebilir bir LP'dir, D'nin candidate-bazlı Big-M reifikasyonları x_{π} 'ye göre her zaman feasible bir beat/rank ataması bulur). Dolayısıyla (t, x, gap) üzerindeki ortak fizibilite kümesi build_model_m4 ile TAMAMEN AYNI — bu modelde bulunan HER feasible (t, x, gap) noktası tam modelde

de feasible'dır. Bu bir GEVŞETME değil, feasibility'ye hiç katkısı olmayan bir alt-sistemin ÇIKARILMASI — bulunan bir çözüm warm-start/local-branching tohumu olarak tam modele TAŞINABİLİR.

Kapsam: `add_flight_time_variables` + `add_b_constraints` (B) + `add_a_constraints` (A) + `add_g_constraints` (G) + `add_e1_constraints` (E1) + `add_e2_constraints` (E2) + `add_f_constraints` (F, rezidüel kapasite dahil). Reward objective YOK — çağırın taraf bir amaç ekler (M5c'de `add_min_deviation_objective` , min-sapma feasibility amacı).

Test kapsamı (`tests/solve/test_build_feasibility_model.py`): C/D'ye ait hiçbir bileşenin (`s / beat / beaten / rank_onehot`) oluşmadığı, A/B/E1/E2/F/G'nin tümünün mevcut olduğu, ve fixture'da min-sapma amacıyla gerçekten OPTIMAL bir çözüme ulaştığı doğrulanıyor.